TCP

client.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define BUFFER_SIZE 1024
int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE] = {0};
    char message[BUFFER_SIZE];
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("\n Socket creation error \n");
        return -1;
    }
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <=
0) {
        printf("\nInvalid address/ Address not supported \n");
        return -1;
    }
    if (connect(sock, (struct sockaddr *)&serv_addr,
sizeof(serv_addr)) < 0) {
        printf("\nConnection Failed \n");
        return -1;
    }
    printf("Connected to server. Type 'exit' to quit.\n");
    while(1) {
        printf("You: ");
        fgets(message, BUFFER_SIZE, stdin);
        message[strcspn(message, "\n")] = 0;
        send(sock, message, strlen(message), 0);
        if (strncmp(message, "exit", 4) == 0) {
            printf("Exiting...\n");
            break;
        }
        memset(buffer, 0, BUFFER_SIZE);
        int valread = read(sock, buffer, BUFFER_SIZE);
        printf("Server: %s\n", buffer);
    }
    close(sock);
    return 0;
}
```

server.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE] = {0};
    char *response = "Message received by server";
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);
    if (bind(server_fd, (struct sockaddr *)&address,
sizeof(address)) < 0) {
        perror("Bind failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }
    if (listen(server_fd, 3) < 0) {
        perror("Listen failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }
```

```
    printf("Server listening on port %d...\n", PORT);
    if ((new_socket = accept(server_fd, (struct sockaddr
*)&address, (socklen_t*)&addrlen)) < 0) {
        perror("Accept failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }
    while(1) {
        memset(buffer, 0, BUFFER_SIZE);
        int valread = read(new_socket, buffer, BUFFER_SIZE);
        if (valread <= 0) {
            printf("Client disconnected.\n");
            break;
        }
        printf("Client: %s\n", buffer);
        send(new_socket, response, strlen(response), 0);
        printf("Response sent to client.\n");
        if (strncmp(buffer, "exit", 4) == 0) {
            printf("Exit signal received. Shutting down.\n");
            break;
        }
    }
    close(new_socket);
    close(server_fd);
    return 0;
}
```

```
client O/P
Connected to server. Type 'exit' to quit.
You: Hi
Server: Message received by server
You: Hello
Server: Message received by server
You: How are you?
Server: Message received by server
You: exit
Exiting...

server O/P
Server listening on port 8080...
Client: Hi
Response sent to client.
Client: Hello
Response sent to client.
Client: How are you?
Response sent to client.
Client: exit
Response sent to client.
Exit signal received. Shutting down.
```

UDP

client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>
#define SERVER_PORT 12345
#define BUFFER_SIZE 1024

int main() {
    int sockfd;
    char buffer[BUFFER_SIZE];
    struct sockaddr_in server_addr;
    socklen_t addr_len = sizeof(server_addr);
    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(SERVER_PORT);
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    while (1) {
        printf("Enter message: ");
        fgets(buffer, BUFFER_SIZE, stdin);
        buffer[strcspn(buffer, "\n")] = 0;
        sendto(sockfd, buffer, strlen(buffer), 0, (struct
sockaddr *)&server_addr, addr_len);
        if (strcmp(buffer, "exit") == 0) {
            printf("Exiting client...\n");
            break;
        }
        int n = recvfrom(sockfd, buffer, BUFFER_SIZE, 0, NULL,
```

```
NULL);
        if (n < 0) {
            perror("Receive failed");
            continue;
        }
        buffer[n] = '\0';
        printf("Server response: %s\n", buffer);
    }
    close(sockfd);
    return 0;
}
```

server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>
#define SERVER_PORT 12345
#define BUFFER_SIZE 1024
int main() {
    int sockfd;
    char buffer[BUFFER_SIZE];
    struct sockaddr_in server_addr, client_addr;
    socklen_t client_addr_len = sizeof(client_addr);
    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(SERVER_PORT);
    if (bind(sockfd, (const struct sockaddr *)&server_addr,
sizeof(server_addr)) < 0) {
        perror("Bind failed");
        close(sockfd);
        exit(EXIT_FAILURE);
    }
    printf("UDP Server listening on port %d...\n", SERVER_PORT);
    while (1) {
        int n = recvfrom(sockfd, buffer, BUFFER_SIZE, 0, (struct
sockaddr *)&client_addr, &client_addr_len);
        if (n < 0) {
            perror("Receive failed");
            continue;
        }
        buffer[n] = '\0';
        if (strcmp(buffer, "exit") == 0) {
            printf("Exit Signal Received, Shutting down
server...\n");
            break;
        }
        printf("Received from client: %s\n", buffer);
        char response[BUFFER_SIZE];
        printf("Enter message: ");
        fgets(response, BUFFER_SIZE, stdin);
        response[strcspn(response, "\n")] = 0;
        sendto(sockfd, response, strlen(response), 0, (struct
sockaddr *)&client_addr, client_addr_len);
    }
    close(sockfd);
    return 0;
}
```

```
client O/P
Enter message: Hi
Server response: Hello
Enter message: How are you?
Server response: Fine
Enter message: exit
Exiting client...

server O/P
UDP Server listening on port 12345...
Received from client: Hi
Enter message: Hello
Received from client: How are you?
Enter message: Fine
Exit Signal Received, Shutting down server...
```

Sliding Window Protocols

Go Back N

protocol.h

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/time.h>
#define PORT 8080
#define WINDOW_SIZE 4
#define TOTAL_PACKETS 10
#define TIMEOUT_SEC 2

typedef struct {
    int seq_num;
    char data[1024];
} Packet;

typedef struct {
    int ack_num;
} AckPacket;
```

client.c

```c
#include "protocol.h"

int main() {
    int sockfd;
    struct sockaddr_in server_addr;
    int base = 0;
    int next_seq = 0;
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr = INADDR_ANY;
    struct timeval tv;
    tv.tv_sec = TIMEOUT_SEC;
    tv.tv_usec = 0;
    setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, &tv,
sizeof(tv));
    printf("Starting GBN Sender (Window Size: %d)\n",
WINDOW_SIZE);
    while (base < TOTAL_PACKETS) {
        while (next_seq < base + WINDOW_SIZE && next_seq <
TOTAL_PACKETS) {
            Packet p;
            p.seq_num = next_seq;
            sprintf(p.data, "Payload %d", next_seq);
            sendto(sockfd, &p, sizeof(p), 0, (struct
sockaddr*)&server_addr, sizeof(server_addr));
            printf("--> Sent Packet %d\n", next_seq);
            next_seq++;
        }
        AckPacket ack;
        if (recvfrom(sockfd, &ack, sizeof(ack), 0, NULL, NULL) >
0) {
            printf("<-- Received ACK %d\n", ack.ack_num);
            if (ack.ack_num >= base) {
                base = ack.ack_num + 1;
            }
        } else {
            printf("!!! Timeout! Resending from Packet %d\n",
base);
            next_seq = base;
        }
    }
    printf("All packets sent and acknowledged successfully.\n");
    close(sockfd);
    return 0;
}
```

server.c

```c
#include "protocol.h"
#include <time.h>

int main() {
    int sockfd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_len = sizeof(client_addr);
    int expected_seq = 0;
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
```

```
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);
    bind(sockfd, (struct sockaddr*)&server_addr,
sizeof(server_addr));
    srand(time(NULL));
    printf("Receiver listening on port %d...\n", PORT);
    while (expected_seq < TOTAL_PACKETS) {
        Packet p;
        recvfrom(sockfd, &p, sizeof(p), 0, (struct
sockaddr*)&client_addr, &addr_len);
        if (rand() % 5 == 0) {
            printf("Dropped Packet %d (Simulated Loss)\n",
p.seq_num);
            continue;
        }
        if (p.seq_num == expected_seq) {
            printf("Received Packet %d. Sending ACK %d\n",
p.seq_num, expected_seq);
            AckPacket ack;
            ack.ack_num = expected_seq;
            sendto(sockfd, &ack, sizeof(ack), 0, (struct
sockaddr*)&client_addr, addr_len);
            expected_seq++;
        } else {
            printf("Out of order Packet %d ignored. Expected
%d\n", p.seq_num, expected_seq);
        }
    }
    printf("All packets received. Exiting.\n");
    close(sockfd);
    return 0;
}
```

client O/P

```
Starting GBN Sender (Window Size: 4)
--> Sent Packet 0
--> Sent Packet 1
--> Sent Packet 2
--> Sent Packet 3
<-- Received ACK 0
--> Sent Packet 4
<-- Received ACK 1
--> Sent Packet 5
<-- Received ACK 2
--> Sent Packet 6
<-- Received ACK 3
--> Sent Packet 7
!!! Timeout! Resending from Packet 4
--> Sent Packet 4
--> Sent Packet 5
--> Sent Packet 6
--> Sent Packet 7
!!! Timeout! Resending from Packet 4
--> Sent Packet 4
--> Sent Packet 5
--> Sent Packet 6
--> Sent Packet 7
<-- Received ACK 4
--> Sent Packet 8
!!! Timeout! Resending from Packet 5
--> Sent Packet 5
--> Sent Packet 6
--> Sent Packet 7
--> Sent Packet 8
!!! Timeout! Resending from Packet 5
--> Sent Packet 5
--> Sent Packet 6
--> Sent Packet 7
--> Sent Packet 8
<-- Received ACK 5
--> Sent Packet 9
<-- Received ACK 6
!!! Timeout! Resending from Packet 7
--> Sent Packet 7
--> Sent Packet 8
--> Sent Packet 9
<-- Received ACK 7
<-- Received ACK 8
<-- Received ACK 9
All packets sent and acknowledged successfully.
```

server O/P

```
Receiver listening on port 8080...
Received Packet 0. Sending ACK 0
Received Packet 1. Sending ACK 1
Received Packet 2. Sending ACK 2
Received Packet 3. Sending ACK 3
```

```
Dropped Packet 4 (Simulated Loss)
Dropped Packet 5 (Simulated Loss)
Out of order Packet 6 ignored. Expected 4
Dropped Packet 7 (Simulated Loss)
Dropped Packet 4 (Simulated Loss)
Out of order Packet 5 ignored. Expected 4
Out of order Packet 6 ignored. Expected 4
Out of order Packet 7 ignored. Expected 4
Received Packet 4. Sending ACK 4
Dropped Packet 5 (Simulated Loss)
Out of order Packet 6 ignored. Expected 5
Out of order Packet 7 ignored. Expected 5
Dropped Packet 8 (Simulated Loss)
Dropped Packet 5 (Simulated Loss)
Out of order Packet 6 ignored. Expected 5
Out of order Packet 7 ignored. Expected 5
Out of order Packet 8 ignored. Expected 5
Received Packet 5. Sending ACK 5
Received Packet 6. Sending ACK 6
Dropped Packet 7 (Simulated Loss)
Out of order Packet 8 ignored. Expected 7
Out of order Packet 9 ignored. Expected 7
Received Packet 7. Sending ACK 7
Received Packet 8. Sending ACK 8
Received Packet 9. Sending ACK 9
All packets received. Exiting.
```

Selective Repeat

client.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>
#define PORT 8080
#define WINDOW_SIZE 4
#define TOTAL_PACKETS 10
int main() {
    int sockfd;
    struct sockaddr_in server_addr;
    bool acked[TOTAL_PACKETS] = {false};
    int base = 0, ack_num;
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
    struct timeval tv = {1, 0};
    setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, &tv,
sizeof(tv));
    while (base < TOTAL_PACKETS) {
        for (int i = base; i < base + WINDOW_SIZE && i <
TOTAL_PACKETS; i++) {
            if (!acked[i]) {
                printf("Sending/Retransmitting Packet: %d\n",
i);
                sendto(sockfd, &i, sizeof(i), 0, (struct
sockaddr*)&server_addr, sizeof(server_addr));
            }
        }
        while (1) {
            if (recvfrom(sockfd, &ack_num, sizeof(ack_num), 0,
NULL, NULL) > 0) {
                printf("Received ACK for: %d\n", ack_num);
                acked[ack_num] = true;
                while (base < TOTAL_PACKETS && acked[base]) {
                    base++;
                }
            } else {
                printf("Timeout! Checking window...\n");
                break;
            }
        }
    }
    printf("Success: All packets acknowledged.\n");
    close(sockfd);
    return 0;
}
```

server.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <arpa/inet.h>
#define PORT 8080
#define WINDOW_SIZE 4
#define TOTAL_PACKETS 10
typedef struct {
    int seq_num;
    bool received;
} ReceiverWindow;
int main() {
    int sockfd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_size = sizeof(client_addr);
    ReceiverWindow buffer[TOTAL_PACKETS] = {0};
    int window_base = 0;
    int incoming_pkt;
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr = INADDR_ANY;
    bind(sockfd, (struct sockaddr*)&server_addr,
sizeof(server_addr));
    printf("Selective Repeat Server listening...\n");
    while (window_base < TOTAL_PACKETS) {
        recvfrom(sockfd, &incoming_pkt, sizeof(incoming_pkt), 0,
(struct sockaddr*)&client_addr, &addr_size);
        printf("Received Packet: %d ", incoming_pkt);
        if (incoming_pkt >= window_base && incoming_pkt <
window_base + WINDOW_SIZE) {
            buffer[incoming_pkt].received = true;
            sendto(sockfd, &incoming_pkt, sizeof(incoming_pkt),
0, (struct sockaddr*)&client_addr, addr_size);
            printf("[ACK Sent]\n");
            while (window_base < TOTAL_PACKETS &&
buffer[window_base].received) {
                window_base++;
            }
        } else {
            printf("[Ignored - Outside Window]\n");
        }
    }
    return 0;
}
```

```
client O/P
Sending/Retransmitting Packet: 0
Sending/Retransmitting Packet: 1
Sending/Retransmitting Packet: 2
Sending/Retransmitting Packet: 3
Received ACK for: 0
Received ACK for: 1
Received ACK for: 2
Received ACK for: 3
Timeout! Checking window...
Sending/Retransmitting Packet: 4
Sending/Retransmitting Packet: 5
Sending/Retransmitting Packet: 6
Sending/Retransmitting Packet: 7
Received ACK for: 4
Received ACK for: 5
Received ACK for: 6
Received ACK for: 7
Timeout! Checking window...
Sending/Retransmitting Packet: 8
Sending/Retransmitting Packet: 9
Received ACK for: 8
Received ACK for: 9
Timeout! Checking window...
Success: All packets acknowledged.

server O/P
Selective Repeat Server listening...
Received Packet: 0 [ACK Sent]
Received Packet: 1 [ACK Sent]
Received Packet: 2 [ACK Sent]
Received Packet: 3 [ACK Sent]
Received Packet: 4 [ACK Sent]
Received Packet: 5 [ACK Sent]
Received Packet: 6 [ACK Sent]
Received Packet: 7 [ACK Sent]
Received Packet: 8 [ACK Sent]
Received Packet: 9 [ACK Sent]
```

Stop and Wait

client.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>

int main() {
    int sockfd;
    struct sockaddr_in server_addr;
    char buffer[1024];
    struct timeval tv;
    tv.tv_sec = 2;
    tv.tv_usec = 0;
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, (const
char*)&tv, sizeof(tv));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(8080);
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
    int frame_to_send = 0;
    while (frame_to_send < 5) {
        char msg[20];
        sprintf(msg, "Frame %d", frame_to_send);
        printf("Sending: %s\n", msg);
        sendto(sockfd, msg, strlen(msg) + 1, 0, (struct
sockaddr*)&server_addr, sizeof(server_addr));
        if (recvfrom(sockfd, buffer, 1024, 0, NULL, NULL) < 0) {
            printf("Timeout! Resending %s...\n", msg);
        } else {
            printf("Received: %s\n", buffer);
            frame_to_send++;
        }
        sleep(1);
    }
    close(sockfd);
    return 0;
}
```

server.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>

int main() {
    int sockfd;
    struct sockaddr_in server_addr, client_addr;
    char buffer[1024];
    socklen_t addr_size;
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(8080);
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
    bind(sockfd, (struct sockaddr*)&server_addr,
sizeof(server_addr));
    addr_size = sizeof(client_addr);
    printf("Receiver is ready to receive frames...\n");
    int expected_frame = 0;
    while (expected_frame < 5) {
        recvfrom(sockfd, buffer, 1024, 0, (struct
sockaddr*)&client_addr, &addr_size);
        printf("Received: %s\n", buffer);
        printf("Sending ACK for frame %d\n", expected_frame);
        sendto(sockfd, "ACK", 4, 0, (struct
sockaddr*)&client_addr, addr_size);
        expected_frame++;
    }
    close(sockfd);
    return 0;
}
```

```
client O/P
Sending: Frame 0
Received: ACK
Sending: Frame 1
Received: ACK
Sending: Frame 2
Received: ACK
Sending: Frame 3
Received: ACK
Sending: Frame 4
Received: ACK

server O/P
Receiver is ready to receive frames...
Received: Frame 0
Sending ACK for frame 0
Received: Frame 1
Sending ACK for frame 1
Received: Frame 2
Sending ACK for frame 2
Received: Frame 3
Sending ACK for frame 3
Received: Frame 4
Sending ACK for frame 4
```

DVR Algorithm

main.c

```c
#include <stdio.h>
#define MAX_NODES 20
#define INF 9999
struct RoutingTable {
    unsigned dist[MAX_NODES];
    unsigned next_hop[MAX_NODES];
} rt[MAX_NODES];
int main() {
    int cost_matrix[MAX_NODES][MAX_NODES];
    int nodes, i, j, k, update_count = 0;
    printf("Enter the number of nodes: ");
    scanf("%d", &nodes);
    printf("\nEnter the cost matrix (use %d for infinity/no
direct link):\n", INF);
    for (i = 0; i < nodes; i++) {
        for (j = 0; j < nodes; j++) {
            scanf("%d", &cost_matrix[i][j]);
            if (i == j) {
                cost_matrix[i][i] = 0;
            }
            rt[i].dist[j] = cost_matrix[i][j];
            rt[i].next_hop[j] = j;
        }
    }
    do {
        update_count = 0;
        for (i = 0; i < nodes; i++) {
            for (j = 0; j < nodes; j++) {
                for (k = 0; k < nodes; k++) {
                    if (rt[i].dist[j] > cost_matrix[i][k] +
rt[k].dist[j]) {
                        rt[i].dist[j] = cost_matrix[i][k] +
rt[k].dist[j];
                        rt[i].next_hop[j] = k;
                        update_count++;
                    }
                }
            }
        }
    } while (update_count != 0);
    printf("\n--- Final Routing Tables ---\n");
    for (i = 0; i < nodes; i++) {
        printf("\nRouting Table for Router %d:\n", i + 1);
        printf("Destination\tNext Hop\tDistance\n");
        for (j = 0; j < nodes; j++) {
            printf("%d\t\t%d\t\t%d\n", j + 1, rt[i].next_hop[j]
+ 1, rt[i].dist[j]);
        }
    }
    return 0;
}
```

```
O/P
Enter the number of nodes: 3

Enter the cost matrix (use 9999 for infinity/no direct link):
0 2 9999
2 0 1
9999 1 0

--- Final Routing Tables ---

Routing Table for Router 1:
Destination     Next Hop        Distance
1               1               0
2               2               2
3               2               3

Routing Table for Router 2:
Destination     Next Hop        Distance
1               1               2
2               2               0
3               3               1

Routing Table for Router 3:
Destination     Next Hop        Distance
1               2               3
2               2               1
3               3               0
```

Leaky Bucket Algorithm

main.c

```c
#include <stdio.h>

int main() {
    int incoming, outgoing, bucket_size, n, store = 0, dropped =
0;
    printf("Enter the bucket capacity: ");
    scanf("%d", &bucket_size);
    printf("Enter the outgoing rate (leak rate): ");
    scanf("%d", &outgoing);
    printf("Enter the number of time intervals to simulate: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("\n--- Time Interval %d ---\n", i + 1);
        printf("Enter the incoming packet size: ");
        scanf("%d", &incoming);
        if (incoming <= (bucket_size - store)) {
            store += incoming;
            printf("Accepted %d packets.\n", incoming);
        } else {
            dropped = incoming - (bucket_size - store);
            printf("Bucket overflow! Dropped %d packets.\n",
dropped);
            store = bucket_size;
        }
        if (store >= outgoing) {
            store -= outgoing;
            printf("Leaked %d packets. ", outgoing);
        } else {
            printf("Leaked %d packets. ", store);
            store = 0;
        }
        printf("Current bucket status: %d / %d\n", store,
bucket_size);
    }
    printf("\n--- Processing remaining packets in the bucket ---
\n");
    while (store > 0) {
        if (store >= outgoing) {
            store -= outgoing;
            printf("Leaked %d packets. ", outgoing);
        } else {
            printf("Leaked %d packets. ", store);
            store = 0;
        }
        printf("Current bucket status: %d / %d\n", store,
bucket_size);
    }
    printf("\nBucket is empty. Simulation complete.\n");
    return 0;
}
```

```
O/P
Enter the bucket capacity: 100
Enter the outgoing rate (leak rate): 20
Enter the number of time intervals to simulate: 3

--- Time Interval 1 ---
Enter the incoming packet size: 40
Accepted 40 packets.
Leaked 20 packets. Current bucket status: 20 / 100

--- Time Interval 2 ---
Enter the incoming packet size: 90
Bucket overflow! Dropped 10 packets.
Leaked 20 packets. Current bucket status: 80 / 100

--- Time Interval 3 ---
Enter the incoming packet size: 10
Accepted 10 packets.
Leaked 20 packets. Current bucket status: 70 / 100

--- Processing remaining packets in the bucket ---
Leaked 20 packets. Current bucket status: 50 / 100
Leaked 20 packets. Current bucket status: 30 / 100
Leaked 20 packets. Current bucket status: 10 / 100
Leaked 10 packets. Current bucket status: 0 / 100

Bucket is empty. Simulation complete.
```

FTP

client.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define BUFFER_SIZE 1024
int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE] = {0};
    char filename[256];
    FILE *file;
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("\n Socket creation error \n");
        return -1;
    }
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <=
0) {
        printf("\nInvalid address/ Address not supported \n");
        return -1;
    } if (connect(sock, (struct sockaddr *)&serv_addr,
sizeof(serv_addr)) < 0) {
        printf("\nConnection Failed \n");
        return -1;
    }
    printf("Connected to the FTP Server.\n");
    printf("Enter the name of the file you want to download: ");
    scanf("%s", filename);
    send(sock, filename, strlen(filename), 0);
    int bytes_received = recv(sock, buffer, BUFFER_SIZE, 0);
    if (strncmp(buffer, "ERROR:", 6) == 0) {
        printf("%s\n", buffer);
    } else {
        char new_filename[300];
        snprintf(new_filename, sizeof(new_filename),
"downloaded_%s", filename);
        file = fopen(new_filename, "wb");
        if (file == NULL) {
            perror("Could not create local file");
            return -1;
        }
        fwrite(buffer, 1, bytes_received, file);
        while ((bytes_received = recv(sock, buffer, BUFFER_SIZE,
0)) > 0) {
            fwrite(buffer, 1, bytes_received, file);
        }
        printf("File downloaded successfully as '%s'.\n",
new_filename);
```

```
        fclose(file);
    }
    close(sock);
    return 0;
}
```

server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE] = {0};
    FILE *file;
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);
    if (bind(server_fd, (struct sockaddr *)&address,
sizeof(address)) < 0) {
        perror("Bind failed");
        exit(EXIT_FAILURE);
    }
    if (listen(server_fd, 3) < 0) {
        perror("Listen failed");
        exit(EXIT_FAILURE);
    }
    printf("FTP Server listening on port %d...\n", PORT);
    if ((new_socket = accept(server_fd, (struct sockaddr
*)&address, (socklen_t*)&addrlen)) < 0) {
        perror("Accept failed");
        exit(EXIT_FAILURE);
    }
    printf("Client connected.\n");
    recv(new_socket, buffer, BUFFER_SIZE, 0);
    printf("Client requested file: %s\n", buffer);
    file = fopen(buffer, "rb");
    if (file == NULL) {
        char *error_msg = "ERROR: File not found.";
        send(new_socket, error_msg, strlen(error_msg), 0);
        printf("File not found. Error message sent to client.
\n");
    } else {
        printf("Sending file...\n");
        int bytes_read;
        while ((bytes_read = fread(buffer, 1, BUFFER_SIZE,
file)) > 0) {
            send(new_socket, buffer, bytes_read, 0);
        }
        fclose(file);
        printf("File transfer complete.\n");
    }
    close(new_socket);
    close(server_fd);
    return 0;
}
```

```
O/P client
Connected to the FTP Server.
Enter the name of the file you want to download: test.txt
File downloaded successfully as 'downloaded_test.txt'.

O/P server
FTP Server listening on port 8080...
Client connected.
Client requested file: test.txt
Sending file...
File transfer complete.
```